

第3章 字句解析

字句解析は、文字記号の系列である入力ファイルから、プログラムの構文を解析するために不要な文字を取り除いて、意味のある単語 (トークン, token) の系列に変換する。単語は、正規言語として定義され、切り分けは有限状態オートマトンを用いる。

3.1 正規表現

正規表現は指定された語の集合 (言語) を指定する。コンパイラにおいては、トークンの種類を正確に指定するために用いる。予約語、識別子、定数、文字列リテラル、区切り子を指定する正規表現を決める。

識別子 (identifier) として認められる文字系列は、「非数字または被数字の後に非数字か数字を複数個接続した系列」として定義される。数字文字 Dec, 非数字文字 NDec, 識別子は ID は次の正規表現で定義することができる。

$$\begin{aligned}\text{Dec} &::= 0|1\cdots|9 \\ \text{NDec} &::= _|\text{a}|\cdots|\text{z}|\text{A}|\cdots|\text{Z} \\ \text{ID} &::= \text{NDec}(\text{NDec}|\text{Dec})^*\end{aligned}$$

10 進数定数 Decimal は以下のように定義できる。

$$\begin{aligned}\text{NZDec} &::= 1|\cdots|9 \\ \text{Decimal} &::= \text{NZDec}\text{Dec}^*\end{aligned}$$

正規表現は次のように定義される。

定義 3.1. 集合 Σ 上の**正規表現**は以下の再帰的定義によって与えられる表現である。

- (1) $\varepsilon, \emptyset$ は正規表現である。
- (2) Σ の要素 $a \in \Sigma$ は正規表現である。
- (3) R_1, R_2 が正規表現であるとき、 $R_1|R_2, R_1 R_2$ は正規表現である。
- (4) R が正規表現であるとき、 R^* は正規表現である。

「再帰的定義」は、自らの定義の中に自身が含まれている定義である。上の定義では、(3),(4)において、正規表現の定義に正規表現自体が使われている。ここでは、定義しようとしている表現よりも短い表現が定義の中で使われているため、うまく定義されていることに注意しよう。例え

ば, $\Sigma - \{a, b\}$ に対して $(ab)|(b^*)$ が Σ 上の正規表現であるためには, (3) から ab と b^* が正規表現でなければならない. ab が正規表現であるためには, (3) から a, b が正規表現でなければならない. $a, b \in \Sigma$ なので, a, b は (2) により正規表現である. b^* が正規表現であるためには b が正規表現でなければならないが, $b \in \Sigma$ なので正規表現である. したがって, $a|(b^*)$ は正規表現である. 括弧はまとまりを表すために適宜用いる. 演算の結合の順番は強い順に $*$, 接続, $|$ とする. $ab|b^*$ は $(ab)|(b^*)$ を表す.

定義 3.2. Σ 上の正規表現 R に対して, R が表す系列の集合 $\llbracket R \rrbracket \subseteq \Sigma^*$ を以下のように再帰的に定める.

- (1) $\llbracket \varepsilon \rrbracket = \{\varepsilon\}, \llbracket \emptyset \rrbracket = \emptyset$
- (2) $a \in \Sigma$ に対して $\llbracket a \rrbracket = \{a\}$
- (3) $\llbracket R_1 R_2 \rrbracket = \llbracket R_1 \rrbracket \cdot \llbracket R_2 \rrbracket, \llbracket R_1 | R_2 \rrbracket = \llbracket R_1 \rrbracket \cup \llbracket R_2 \rrbracket$
- (4) $\llbracket R^* \rrbracket = \llbracket R \rrbracket^*$

ここで右辺に現れる $\cdot, *$ は??節で定義した集合に対する接続と閉包を表す.

従って, $ab|b^*$ という正規表現が表す Σ の系列の集合は以下ようになる.

$$\begin{aligned}
 \llbracket ab|b^* \rrbracket &= \llbracket ab \rrbracket \cup \llbracket b^* \rrbracket \\
 &= \llbracket a \rrbracket \llbracket b \rrbracket \cup \llbracket b \rrbracket^* \\
 &= (\{a\} \cdot \{b\}) \cup \{b\}^* \\
 &= \{ab\} \cup \{\varepsilon, b, bb, bbb, \dots\}
 \end{aligned}$$

プログラミング言語のトークンは正規表現で定義される.

3.2 有限オートマトン

正規表現によって表現される文字列を認識するメカニズムは有限オートマトンである. 有限オートマトンは, 有限の状態を持つ状態遷移機械であり, ある状態 q_1 から次の状態 q_2 へ遷移するとき Σ の要素 a を読む. この遷移を $q_1 \xrightarrow{a} q_2$ と書く. 初期状態から, 受理状態に至る遷移の系列で受理する文字系列がオートマトンが受理する語である. q_0 が初期状態で, $q_0 \xrightarrow{a_1} q_1, q_1 \xrightarrow{a_2} q_2, \dots, q_{n-1} \xrightarrow{a_n} q_n$ という遷移があり, q_n が受理状態であるとき, 語 $w = a_1 a_2 \dots a_n$ をオートマトンが受理するという.

定義 3.3. 有限オートマトン は以下の5項組で定義される. $\langle Q, \Sigma, \delta, q_0, F \rangle$ ここで, Q は状態の有限集合, δ は遷移関係, $q_0 \in Q$ は初期状態, $F \subseteq Q$ は受理状態集合である.

δ は3つ組 (q, a, q') の集合であり, q から a を読んで q' に遷移することを示す. 遷移関係は $(q, a, q') \in \delta$ を $q \xrightarrow{a} q'$ のように表す. 遷移関係 δ を Σ^* に以下のように拡張した関係 $q \xRightarrow{s} q'$ に拡張する.

$$\begin{aligned}
 q &\xRightarrow{\varepsilon} q \\
 q &\xRightarrow{as} q' \quad \text{ここで, } q \xrightarrow{a} q'', q'' \xRightarrow{s} q'
 \end{aligned}$$

定義 3.4. 有限オートマトン $A = \langle Q, \Sigma, \delta, q_0, F \rangle$ に対して $L(A) = \{w \mid q_0 \xrightarrow{w} q_f, q_f \in F\}$ を A の受理言語という.

有限オートマトンの受理言語と正規表現が表す言語には以下のようなよく知られた関係がある.

定理 3.1. • 正規表現 R に対して $\llbracket R \rrbracket$ を受理する有限オートマトン A_R が存在する.

• 有限オートマトン A に対して $L(A) = \llbracket R \rrbracket$ となる正規表現 R が存在する.

このことから、正規表現と有限オートマトンは表現能力が等しいことがわかる. また、有限オートマトンは、受理言語の積、和、否定に関して閉じていることが知られている.

定理 3.2. • 有限オートマトン A_1, A_2 に対して、 $L(A_3) = L(A_1) \cap L(A_2)$, $L(A_4) = L(A_1) \cup L(A_2)$ を満たす有限オートマトン A_3, A_4 が存在する.

• 有限オートマトン A に対して、 $L(A_c) = \Sigma^* \setminus L(A)$ を満たす有限オートマトン A_c が存在する.

上記定理の証明については、オートマトンの教科書¹を参照してほしい.

3.3 字句解析の仕組み

字句解析では、原始プログラムを Σ の系列として受取り、トークンごとに切り分けて出力する. トークンの種類に応じてオートマトンを構成する. それぞれのオートマトンを A_{kw} , A_{id} , A_{cnst} , A_{str} , A_{del} とする.

字句解析はこれらのオートマトンを図??のように組み合わせて構成される.

¹J. ホップクロフト/J. ウルマン:「オートマトン言語理論 計算論1」訳:野崎昭弘ら 2.5 および 3.2

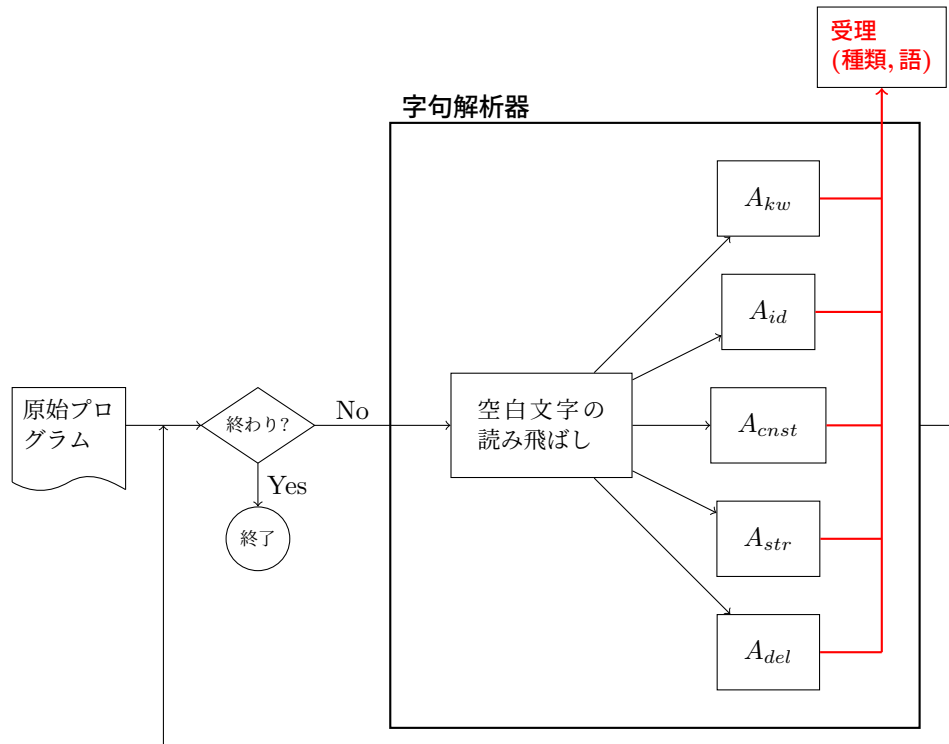


図 3.1: 字句解析器の構成

ステップ 1: 空白、タブコード、改行などの空白文字を読み飛ばす。

ステップ 2: 次の文字を A_{kw} , A_{id} , A_{cnst} , A_{str} , A_{del} に入力する。

ステップ 3: いずれかのオートマトンが受理状態に達するまで、ステップ 2 を繰り返す。

ステップ 4: いずれかのオートマトンが受理状態に達したらそのオートマトンの種類と受理した系列を出力し、すべてのオートマトンの状態を初期状態に戻す。

ステップ 5: 入力の最後に達していなければ、ステップ 1 に戻る。入力の最後に達したら終了

字句解析の実現においては、トークンの全体の和集合を受理するオートマトンを作成し、受理状態がどのトークンによるものかを区別しておくことで字句解析が実現ができる。有限オートマトンが和集合について閉じていることからこのような構成はいつも可能であることが保証されている。

受理状態が2つ以上のオートマトンによる場合は、受理の優先順位を決めておくことで字句解析を行う。予約語と識別子は一般に同じ文字列を受理する可能性がある。その場合は、予約語を優先させる。

字句解析器のプログラミング 字句解析を行うオートマトンを構成するために、正規表現を用いる。正規表現 R を $\llbracket R \rrbracket$ を受理する有限オートマトンに変換して、字句解析器に組み込む。有限オー

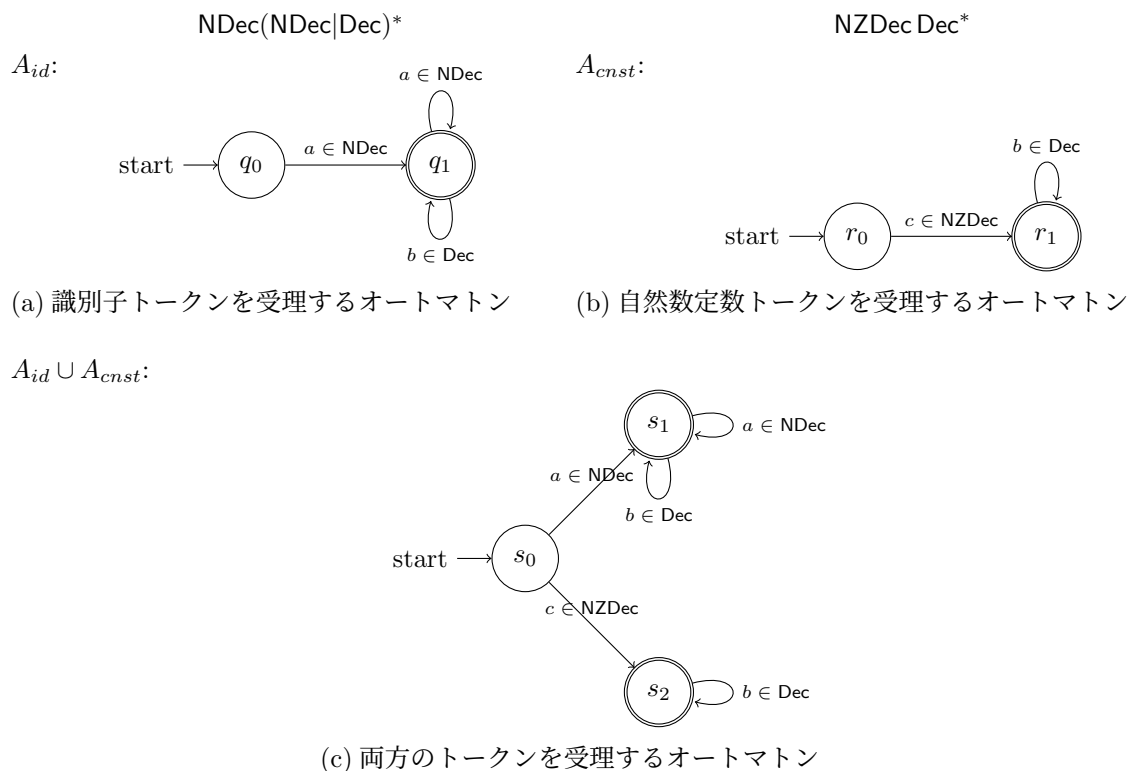


図 3.2: 字句解析の正規表現とオートマトン

トマソンからプログラムへの自動変換は比較的容易に実現できる。字句解析器は、トークンの集合を正規表現で記述するだけで、自動生成することが可能である。

図 3.2 に識別子と自然数定数を受理するオートマトンを示す。いずれかを受理するオートマトン $A_{id \cup cnst}$ は (c) のようになる。 s_1 が受理状態であった場合は識別子、 s_2 が受理状態であった場合は自然数定数である。

3.4 字句解析器の生成

3.4.1 正規表現の記法

オートマトンに対応する動作をさせるために計算機上で正規表現は以下のように記述される。

- ϵ , $\emptyset$ を直接表現する記法はない。
- $a \in \Sigma$ については, "a" のように引用符で囲む。 $s \in \Sigma^+$ についてはまとめて "abc" のように引用符で囲む。
- $R_1|R_2$ については, R_1 の記法 | R_2 の記法。連続した文字コードを持つ場合には, $[X-Y]$ のように途中を省略できる。 $s = a_1a_2 \dots a_n \in \Sigma^+$ に対して $[s]$ は $a_1|a_2|\dots|a_n$ を表す。例えば $[0-9]$ は $0|1|2|3|4|5|6|7|8|9$ を表す。

- Kleene 閉包は * を後ろにつける。[ab]* など
- ラベルの後ろに正規表現を書いて名前をつけることができる。NZDEC [1-9] や DEC [0-9] など。ラベルの参照時は {} をつける。

ラベルの機能を使うと自然数定数は {NZDEC}{DEC}* と記述できる。

NDEC [_a-zA-Z] とすると、識別子名を表す系列は {NDEC}({NDEC}|{DEC})* となる。

3.4.2 字句解析器生成ツール

lex

lex では正規表現とアクション (C 言語) の対を並べて記述し、入力文字列から正規表現で構成されるオートマトンが受理する系列を抽出して、対応するアクションを実行するような C 言語プログラムを生成する。受理した入力系列は yytext という文字列に格納される。

例えば、以下のような仕様ファイル lex1.1 を用意したとする。

```
%%
[A-Za-z]+ {printf("String found:%s\n",yytext);}
[0-9]+    {printf("Digits found:%s\n",yytext);}
.         {printf("Other character:%d\n",yytext[0]);}
%%
```

lex では、ピリオド "." は Σ の任意の文字のいずれかであることを示す。受理の優先度は上に書かれるほど高い。入力の "A" は 1 番めと 3 番めの両方に受理されるが、アクションは 1 番めが実行される。lex は、上のような仕様を受け取って、正規表現にマッチする系列が入力されたとき、アクションを実行する C 言語プログラムを生成する。

```
% lex -o lex1.c lex1.1
% cc -o lex1 lex1.c -ll
```

1 行目で仕様 lex1.1 から lex1.c を生成し、これを 2 行目で実行形式に変換している。

より複雑なプログラムを書くために最初の %% の前に C 言語の include 文や define マクロまたはプロトタイプ宣言、最後の %% の後に C 言語の関数を記述することができる。

```
%{
#include <stdio.h>
void pp(char*);
}%
%%
[A-Za-z]+ {pp("String");}
[0-9]+    {pp("Digits");}
```

```

        {printf("Other character:%d\n",yytext[0]);}
%%
int main(void)
{
    yylex();
    return 0;
}
void pp(char *s) {
    printf("%s found:%s\n",s,yytext);
}

```

sly(python)

Python では sly パッケージ²で字句解析器を生成できる。正規表現は、r に続けて記載する。正規表現で制御に使う文字はバックスラッシュでエスケープする。python における正規表現については <https://docs.python.org/ja/3/howto/regex.html> などを参照せよ。

```

int x;
int f(int y){
    r=3; return x+y;
}
int main(void) {
int x,y; x=6;
{int x; x=7;
y=f(x);}
printf("%02d",x+y);
}

```

上記の C 言語風のプログラムを字句解析をすると以下のようになる。

```

from sly import Lexer

class CalcLexer(Lexer):
    # Set of token names. This is always required
    tokens = { ID, STRING, KWORD, NUMBER, PLUS, MINUS, TIMES,
    DIVIDE, ASSIGN, LPAREN, RPAREN, LBRACE, RBRACE, SEMICOLON, COMMA }

    # String containing ignored characters between tokens
    ignore = ' \t\n'

```

²<https://sly.readthedocs.io/en/latest/sly.html#writing-a-lexer>

```
# Regular expression rules for tokens
ID      = r'[a-zA-Z_][a-zA-Z0-9_]*'
STRING  = r'\".*?\"'
KEYWORD = r'int|return|printf|void'
NUMBER  = r'\d+'
PLUS    = r'\+'
MINUS    = r'\-'
TIMES    = r'\*'
DIVIDE   = r'\/'
ASSIGN   = r'='
LPAREN   = r'\('
RPAREN   = r'\)'
LBRACE   = r'\{'
RBRACE   = r'\}'
SEMICOLON = r';'
COMMA    = r','

if __name__ == '__main__':
    data = '''
    int x;
    int f(int y){
        r=3; return x+y;
    }
    int main(void) {
    int x,y; x=6;
    {int x; x=7;
    y=f(x);}
    printf("%02d",x+y);
    }
    '''

    lexer = CalcLexer()
    for tok in lexer.tokenize(data):
        print('type=%r, value=%r' % (tok.type, tok.value))
```